

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SCHOOL OF TECHNOLOGY

PANDIT DENDAYAL ENERGY UNIVERSITY

ACADEMIC DATABASE MANAGEMENT SYSTEM PROJECT REPORT

BLACKSITE // OPERATOR ARMORY

Next-Generation Game Inventory Management, P2P Marketplace & Scarcity Economy System

---

Course	Database Management Systems (DBMS)
Submitted by	*Ishan Patel, Dhanya Barot, Nishka Shah, Netraa Patel*
Roll No.	*25BCP113, 25BCP119, 25BCP123, 25BCP125*
Submitted to	*Dr. Sonam Nahar*
Academic Year	*[2026-27]*

Table of Contents

1. Executive Summary / Abstract 5
1.1 Tech-Stack Used 6
2. System Architecture & Component Topology 7
2.1 Architectural Tier Breakdown 7
2.2 System Architecture Diagram 7
3. Conceptual Model & ER Diagram ..... **Error! Bookmark not defined.**
3.1 Some Important Entities 9
3.2 ER Diagram Specifications 9
3.3 Relationship Cardinality & Constraints 10
4. Formal Relational Schema & Normalization 12
4.1 Phase 1: Universal Relation and Initial Functional Dependencies 12
4.2 Phase 2: Step-by-Step Normalization 12
4.3 Summary of Normalization 14
5. Complete Executable DDL (Data Definition Language) 15
6. Complete DML & Seed Dataset 19
7. Some Related Queries 21
8. Core Subsystems & Business Logic 27
8.1 Player Equipment and Loadouts 27
8.2 Supply Depot & Vendor Purchasing 27
8.3 Scarcity-Driven P2P Black Market 27
8.4 Direct Player Trading 27
8.5 Scarcity-Driven Luck Royale & Redistribution Engine 27
9. Concurrency Control & Transaction Management 28
9.1 Deadlock Prevention Strategy 28
9.2 Network Idempotency Tokens 28
10. Security, Dual-Gateway Access & Audit Trails 28
11. Overall Takeaway / Conclusion 29

1. Executive Summary / Abstract

The Problem: Managing High-Stakes Virtual Economies

Modern multiplayer shooters and competitive tactical gaming platforms (such as Call of Duty, Counter-Strike 2, and Escape from Tarkov) rely heavily on high performance database architectures. Such systems have to be able to manage millions of dollars of virtual micro-economies, monitor hundreds of thousands of items, avoid race conditions when trading peer-to-peer items, and ensure the scarcity of unique items.

The Solution: Introducing BLACKSITE // ARMORY

BLACKSITE // ARMORY is a relational database management system built specifically to solve these challenges. The platform is designed to cater to real world gaming economy exploits with seamless user experience.

With the effective incorporation of a Peer-to-Peer Black Market, escrow services for bilateral trades, and inventory tracking for individual instances, the platform is extremely adept at handling high volume transactions of digital assets without any data irregularities. This project brings together theoretical concepts in databases with practical production needs, providing an efficient and high performance backend infrastructure for competitive tactical gaming micro-economies.

Core Economy Mechanics

- **Multi-Currency Asset Management:** Tracks both premium and earned game currencies (e.g., COD Points, Salvage) using strictly constrained wallet tables that utilize database-level checks to categorically prevent negative balances and overdrawing.
- **Instance-Based Inventories (The Armory):** Differentiates between master catalog definitions (e.g., a master weapon blueprint) and distinct player-owned instances. This allows the system to track the exact ownership, acquisition date, micro-durability, and status of every individual item currently active in the game world.
- **The Black Market Exchange:** Facilitates a robust, player-driven marketplace where users can list specific item instances for sale, dynamically search active listings, and purchase items from others using various in-game currencies.
- **Bilateral Trade Escrow Engine:** Powers secure, direct player-to-player barter using a dual-confirmation transaction system that locks items and funds securely before executing an exchange.
- **Luck Royale Drop Wheel:** An innovative, scarcity-driven distribution mechanic that manages randomized loot drops while interacting directly with the store and inventory ledgers.

Technical Architecture & Security Protocols

To keep the virtual economy safe and fair, **BLACKSITE // ARMORY** is built on a clean, tightly structured database foundation. We've organized the system using Third Normal Form (3NF) standards—eliminating data conflicts so item attributes and player balances stay perfectly in sync across every server.

To shut down exploits and maintain absolute integrity, the platform relies on two key security layers:

- **Fail-Safe Transactions (ACID Compliance):** Every trade runs inside a protected transaction block that temporarily locks the specific rows being updated (FOR UPDATE). If a player's connection drops mid-trade or someone attempts a simultaneous double-spend, the entire transaction instantly rolls back to its original state.

- **Automated Auditing & Exploit Prevention:** Database-level triggers automatically record high-value transfers, track item provenance across the Black Market, and block suspicious actions on the spot—such as listing the same item twice at once.

By pairing an immersive tactical theme with hardened backend design, BLACKSITE // ARMORY proves how high-stakes virtual economies can remain completely secure behind the scenes.

Key Objectives Achieved

- **ACID Compliance:** Eliminated duplicate spending and inventory duplication exploits through pessimistic row-level locking.
- **Formal Relational Modeling:** Decomposed all entities to satisfy 1NF, 2NF, 3NF, and BCNF with formal mathematical proofs.
- **Scarcity-Driven Economy:** Engineered a dynamic probability formula that mathematically redistributes drop chances when items sell out.
- **Dual-Gateway Security:** Implemented strict role-based separation between administrator maintenance tools and tactical player sessions.
- **Multi-Engine Portability:** Native auto-probing connection pool that transparently switches between XAMPP MariaDB and PostgreSQL without code alteration.

1.1 Tech-Stack Used

> **Application Name:** BLACKSITE // OPERATOR ARMORY
>
> **Domain:** Database Management Systems (DBMS) / Interactive Media
>
> **Classification:** A full-stack academic and production-grade game database system featuring 3NF/BCNF modeling, row-level concurrency control, dual-gateway security, bilateral escrow handshakes, and a scarcity-driven drop engine.

Project Metadata	Specification Details
Application Name	BLACKSITE // OPERATOR ARMORY
Course & Domain	Database Management Systems (DBMS) / Interactive Media
Target Engines	MariaDB 10.4+ / MySQL 8.0+ (XAMPP) & PostgreSQL 14+
Backend Tech Stack	Node.js, Express REST API, Multi-Database Connection Pool
Frontend Tech Stack	React 18, Vite, Three.js Tactical Canvas & 2D Tactical Viewers
Normalization Level	Third Normal Form (3NF) & Boyce-Codd Normal Form (BCNF)
Concurrency Standard	ACID Transactions with Row-Level Locking (SELECT ... FOR UPDATE)
Specialized Module	Scarcity Engine with Atomic Supply Cap & Probability Redistribution
Verification Status	19/19 Unit & Integration Tests Verified (Zero Defect Pass)
Project Workspace	c:\DBMS_Project

2. System Architecture & Component Topology

The application uses a clean 3-Tier Enterprise Architectural Pattern consisting of a Client Presentation Tier, an Application & Business Logic Tier, and a Multi-Engine Database Persistence Tier.

2.1 Architectural Tier Breakdown

- **Presentation Tier (React Frontend):** Single Page Application (SPA) built with React 18 and Vite. Features a Call of Duty tactical aesthetic. Includes real-time loadout builders, animated canvas drop wheel with sound/visual feedback, and responsive inventory inspection.
- **Application Tier (Node.js & Express):** Node.js with Express implementing RESTful endpoints. Contains modular controllers for authentication, operator inventory, store transactions, P2P marketplace, trade escrow, and the scarcity engine. Implements parameterized query sanitization, deadlock prevention algorithms, and transaction wrappers.
- **Persistence Tier (Relational RDBMS):** Dual-dialect relational engine supporting XAMPP MariaDB/MySQL (InnoDB Engine with utf8mb4 collation) and PostgreSQL 14+. The db.js abstraction layer provides automatic parameter mapping ($1 -> ?), ILIKE to LIKE adaptation, boolean normalization, and transactional savepoints.

2.2 System Architecture Diagram

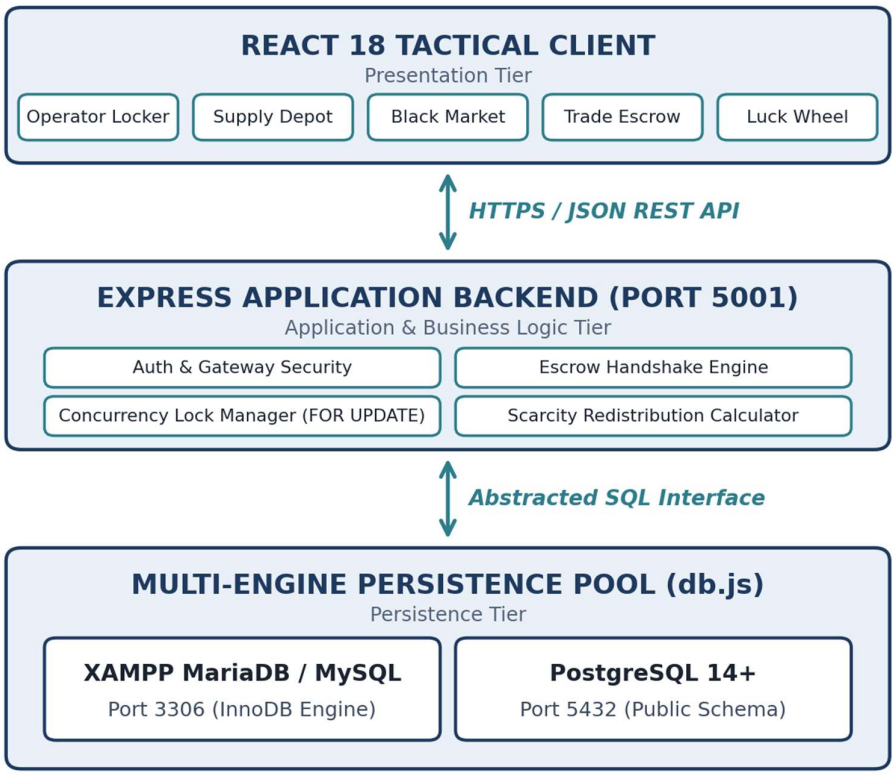


Figure 2.1 — System architecture and component topology

3. Conceptual Model & ER Diagram

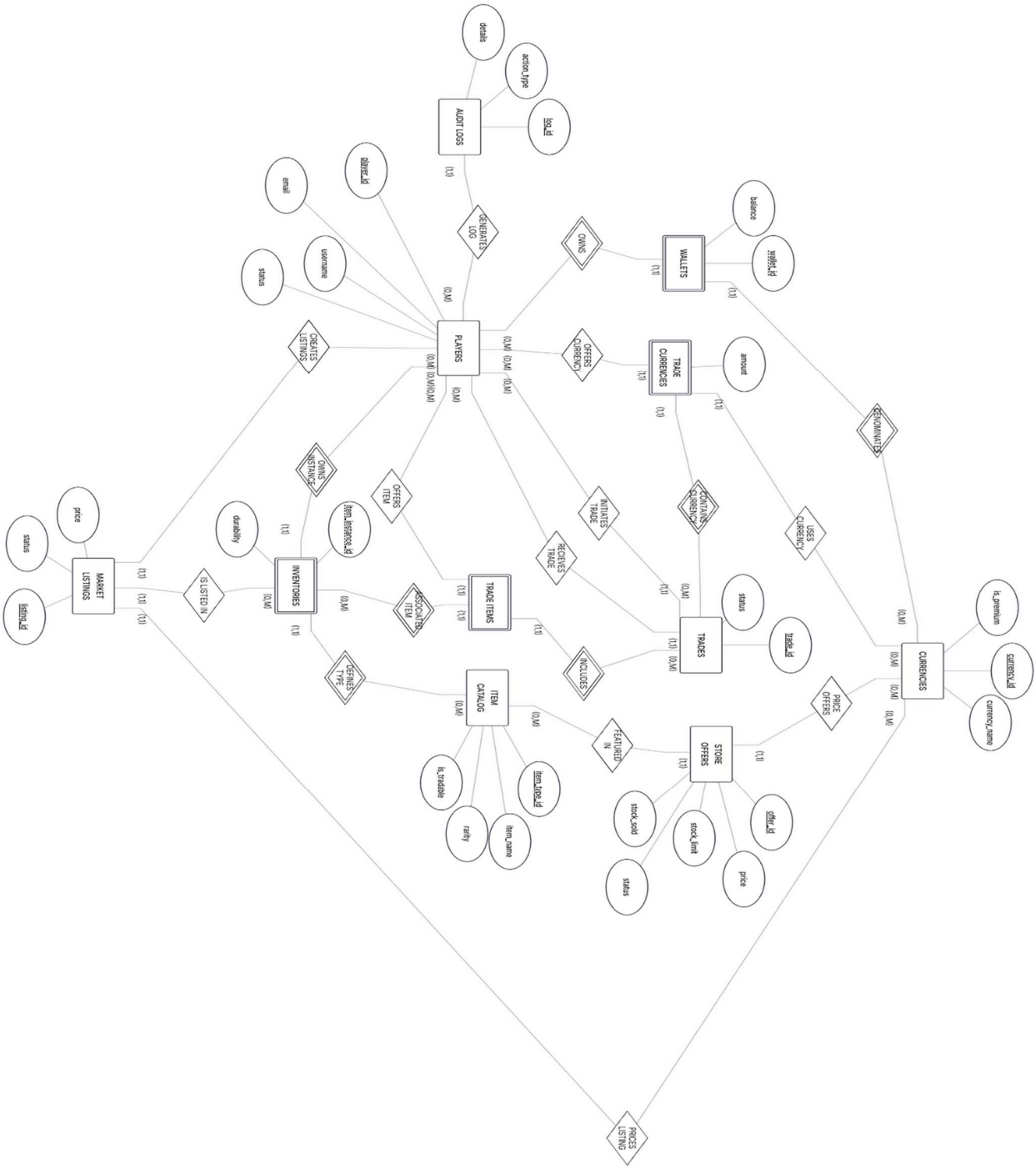


Figure 3.1 — Entity-Relationship Diagram (ERD)

The Entity-Relationship Diagram (ERD) conceptually maps the complete architecture of the virtual economy, detailing the core entities, attributes, primary keys, and their precise participation constraints. The model is structured around three primary functional modules:

3.1 Some Important Entities

1. **1. Currencies & Wallets:** The Currencies entity defines the available economic denominators (both premium and earned), while Wallets acts as the transactional ledger connecting a specific player to an exact currency balance.
2. **2. Item_Catalog & Inventories:** The Item_Catalog stores the master game blueprints, establishing baseline attributes like rarity and base type. The Inventories entity represents the physical realization of these blueprints, tracking distinct, individual asset instances owned by specific players.
3. **3. Trades:** Manages the lifecycle of peer-to-peer bilateral exchanges, linking an initiator and a receiver to determine the active status of an escrow session.
4. **4. Market_Listings & Store_Offers:** These entities handle the commercial aspects of the economy, defining the price, stock limits, and status of items placed for sale by either users or the official developer storefront.
5. **5. Audit_Logs:** A dedicated security entity that records critical economy events and captures JSON state changes to maintain an immutable administrative ledger.

3.2 ER Diagram Specifications

Entity Name	Primary Key	Attributes	Business Function
PLAYERS	`player_id`	username, email, status, created_at	Registered game operators and system administrators.
CURRENCIES	`currency_id`	currency_name, is_premium, symbol, description	Operational scrip: Combat Credits (CR), Blacksite Tokens (BST), etc.
WALLETS	`wallet_id`	player_id, currency_id, balance, updated_at	Financial balances per player per currency denomination.
ITEM_CATALOG	`item_type_id`	item_name, rarity, is_tradable, category, durability, asset_key	Master blueprint definitions of weapons, gear, skins, and crates.
INVENTORIES	`item_instance_id`	player_id, item_type_id, durability, is_equipped, acquired_at	Physical instantiated items owned by players with individual health.
STORE_OFFERS	`offer_id`	item_type_id, currency_id, price, stock_limit, stock_sold, status	Armory vendor listings with strict inventory replenishment limits.
MARKET_LISTINGS	`listing_id`	seller_id, item_instance_id, currency_id, price, status	Peer-to-peer player market listings with escrow locking.
TRADES	`trade_id`	initiator_id, receiver_id, status, created_at, resolved_at	Bilateral player-to player trade negotiation sessions.

Entity Name	Primary Key	Attributes	Business Function
TRADE_ITEMS	`trade_item_id`	trade_id, item_instance_id, player_id	Specific item instances committed to a trade proposal.
TRADE_CURRENCIES	`trade_currency_id`	trade_id, currency_id, player_id, amount	Cash scrip offered as part of a bilateral trade proposal.
AUDIT_LOGS	`log_id`	player_id, action_type, details, created_at	Immutable security ledger recording every financial/inventory mutation.
WHEEL_EVENTS	`event_id`	event_name, currency_id, spin_cost, is_active, created_at	Luck Royale drop wheel promotional banner configuration.
WHEEL_REWARDS	`reward_id`	event_id, item_type_id, base_probability, max_supply, awarded_count	Weighted drop table with strict atomic global supply limits.
WHEEL_SPINS	`spin_id`	player_id, reward_id, item_instance_id, cost_paid, idempotency_key	Historical spin ledger ensuring non repudiation and idempotency.

3.3 Relationship Cardinality & Constraints

Relationship	Cardinality & Participation Constraint
PLAYERS (1) — (0..M) WALLETS	Every wallet belongs to exactly 1 player. A player holds 0 to M wallets (1 per currency).
CURRENCIES (1) — (1..M) WALLETS	Every wallet is denominated by exactly 1 currency. A currency can denominate 1 to M player wallets.
PLAYERS (1) — (0..M) INVENTORIES	Every inventory instance belongs to exactly 1 player. A player possesses 0 to M items.
ITEM_CATALOG (1) — (0..M) INVENTORIES	A catalog blueprint defines 0 to M physical inventory items. Each physical item references exactly 1 blueprint.
ITEM_CATALOG (1) — (0..M) STORE_OFFERS	Every store offer features exactly 1 catalog blueprint. A blueprint can be featured in 0 to M store offers over time.
CURRENCIES (1) — (0..M) STORE_OFFERS	Every store offer is priced in exactly 1 currency. A currency can price 0 to M store offers.
PLAYERS (1) — (0..M) MARKET_LISTINGS	Every market listing is created by exactly 1 seller. A player can create 0 to M market listings.
INVENTORIES (1) — (0..1) MARKET_LISTINGS	An inventory item can be actively listed at most once. (Enforced via status = 'Active' uniqueness constraint). An active listing references exactly 1 inventory instance.
CURRENCIES (1) — (0..M) MARKET_LISTINGS	Every market listing is priced in exactly 1 currency. A currency can price 0 to M market listings.
PLAYERS (2) — (0..M) TRADES	A trade requires exactly 2 distinct players (sender_id != receiver_id). Players can engage in multiple trades as either sender or receiver.

Relationship	Cardinality & Participation Constraint
TRADES (1) — (0..M) TRADE_ITEMS	A trade can involve 0 to M items from both parties, resolved through the associative entity TRADE_ITEMS. Each trade item entry belongs to exactly 1 trade session.
INVENTORIES (1) — (0..M) TRADE_ITEMS	Every trade item entry references exactly 1 physical inventory item instance. An item instance can appear in 0 to M trade item records historically.
PLAYERS (1) — (0..M) TRADE_ITEMS	Every trade item entry is offered by exactly 1 player. A player can offer 0 to M items across various trades.
TRADES (1) — (0..M) TRADE_CURRENCIES	A trade can involve 0 to M currency pledges, resolved through the associative entity TRADE_CURRENCIES. Each trade currency entry belongs to exactly 1 trade session.
CURRENCIES (1) — (0..M) TRADE_CURRENCIES	Every trade currency entry specifies exactly 1 currency type. A currency can be pledged in 0 to M trade currency entries.
PLAYERS (1) — (0..M) TRADE_CURRENCIES	Every trade currency entry is offered by exactly 1 player. A player can offer 0 to M currency pledges across trades.
PLAYERS (1) — (0..M) AUDIT_LOGS	Every audit log entry records an action for exactly 1 player. A player can generate 0 to M audit logs over time.

4. Formal Relational Schema & Normalization

Database Normalization via Functional Dependencies (FDs)

To formally prove that the virtual economy database schema is designed without data redundancy, update anomalies, or insertion anomalies, we apply formal **Functional Dependency (FD)** analysis and relational decomposition up to **Third Normal Form (3NF)**.

4.1 Phase 1: Universal Relation and Initial Functional Dependencies

We begin with a universal relation R containing all attributes of the virtual item economy system:

> **R** = (player_id, username, email, status, currency_id, currency_name, is_premium, wallet_id, balance, item_instance_id, item_type_id, item_name, rarity, is_tradable, durability, offer_id, price, stock_limit, stock_sold, listing_id, trade_id, trade_status, offered_by_player_id, amount, log_id, action_type, details, created_at)

Core Functional Dependencies in R:

No.	Functional Dependency
1	player_id → {username, email, status}
2	currency_id → {currency_name, is_premium}
3	item_type_id → {item_name, rarity, is_tradable}
4	wallet_id → {player_id, currency_id, balance}
5	(player_id, currency_id) → {wallet_id, balance}
6	item_instance_id → {player_id, item_type_id, durability}
7	offer_id → {item_type_id, price, currency_id, stock_limit, stock_sold, status}
8	listing_id → {seller_id, item_instance_id, price, currency_id, status}
9	trade_id → {sender_id, receiver_id, status}
10	(trade_id, item_instance_id) → offered_by_player_id
11	(trade_id, player_id, currency_id) → amount
12	log_id → {player_id, action_type, details, created_at}

4.2 Phase 2: Step-by-Step Normalization

Step 1: Unnormalized Form (UNF) to First Normal Form (1NF)

Condition: All attributes must be atomic; repeating groups or multi-valued attributes are prohibited.

Problem in UNF: A single player owns multiple wallets, multiple item instances, and engages in multiple trades, creating repeating groups within a single row.

Decomposition Action: We isolate repeating groups into independent relations, establishing surrogate primary keys where each attribute is atomic.

Resulting 1NF Relations & FDs:

Relation Schema (primary key underlined)	Functional Dependency
PLAYERS = (player_id, username, email, status)	**FD:** player_id → username, email, status
CURRENCIES = (currency_id, currency_name, is_premium)	**FD:** currency_id → currency_name, is_premium
ITEM_CATALOG = (item_type_id, item_name, rarity, is_tradable)	**FD:** item_type_id → item_name, rarity, is_tradable
WALLETS = (wallet_id, player_id, currency_id, balance)	**FDs:** (player_id, currency_id) → wallet_id, balance
INVENTORIES = (item_instance_id, player_id, item_type_id, durability)	**FD:** item_instance_id → player_id, item_type_id, durability

Step 2: First Normal Form to Second Normal Form (2NF)

Condition: The relation must be in 1NF, and **every non-prime attribute must be fully functionally dependent on the entire primary key** (no partial dependencies on a composite key).

Decomposition Action: We examine tables with composite keys, specifically our associative trade junction tables.

Analysis & Resulting 2NF Relations:

In TRADE_ITEMS, the primary key is a composite key: (trade_id, item_instance_id). The non-prime attribute `offered_by_player_id` depends fully on the combination of both keys:

$$\{(\text{trade_id}, \text{item_instance_id})\} \rightarrow \text{offered_by_player_id}$$

(Since there are no proper subsets of the composite key that determine `offered_by_player_id`, no partial dependency exists).

In TRADE_CURRENCIES, the composite primary key is (trade_id, player_id, currency_id):

$$\{(\text{trade_id}, \text{player_id}, \text{currency_id})\} \rightarrow \text{amount}$$

(Fully dependent on the entire three-way composite key).

Step 3: Second Normal Form to Third Normal Form (3NF)

Condition: The relation must be in 2NF, and **no transitive dependencies** must exist ($X \rightarrow Y$ where Y is non-prime and X is not a superkey). Non-prime attributes cannot determine other non-prime attributes.

Verification Action: We check remaining standalone entity tables to ensure non-key attributes rely strictly on primary keys rather than other non-key fields.

Resulting 3NF Relations:

Relation Schema (primary key underlined)	Functional Dependency
STORE_OFFERS = (offer_id, item_type_id, price, currency_id, stock_limit, stock_sold, status)	**FD:** offer_id → {item_type_id, price, currency_id, stock_limit, stock_sold, status} *(No transitive dependencies;* `item_type_id` *and* `currency_id` *are foreign keys).*
MARKET_LISTINGS = (listing_id, seller_id, item_instance_id, price, currency_id, status)	**FD:** listing_id → {seller_id, item_instance_id, price, currency_id, status}
TRADES = (trade_id, sender_id, receiver_id, status)	**FD:** trade_id → {sender_id, receiver_id, status}
AUDIT_LOGS = (log_id, player_id, action_type, details, created_at)	**FD:** log_id → {player_id, action_type, details, created_at}

4.3 Summary of Normalization

Every relation in our 11-table schema satisfies **3NF**, ensuring that all non-key attributes are directly and solely dependent on their primary candidate keys, completely eliminating redundancy and update anomalies.

5. Complete Executable DDL (Data Definition Language)

The complete executable MySQL Data Definition Language (DDL) script for the BLACKSITE database schema incorporates all primary keys, foreign keys, and domain constraints derived from the final ER diagram. It incorporates table structures, primary keys, foreign key referential integrity constraints with appropriate ON DELETE actions, unique indexes, and relational views.

```
-- ========================================================================
-- BLACKSITE // OPERATOR ARMORY - EXECUTABLE DDL SCHEMA
-- Compatible with XAMPP MariaDB 10.4+ / MySQL 8.0+ & PostgreSQL 14+
-- ========================================================================
CREATE DATABASE IF NOT EXISTS blacksite_armory CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE blacksite_armory;

-- 1. PLAYERS TABLE
CREATE TABLE players (
    player_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    status ENUM('active', 'suspended', 'offline', 'banned') NOT NULL DEFAULT 'active',
    created at DATETIME NOT NULL DEFAULT CURRENT TIMESTAMP
) ENGINE=InnoDB;

-- 2. CURRENCIES TABLE
CREATE TABLE currencies (
    currency_id INT AUTO_INCREMENT PRIMARY KEY,
    currency name VARCHAR(50) NOT NULL UNIQUE,
    is_premium BOOLEAN NOT NULL DEFAULT FALSE,
    symbol VARCHAR(10) NOT NULL DEFAULT 'CR',
    description TEXT
) ENGINE=InnoDB;

-- 3. WALLETS TABLE
CREATE TABLE wallets (
    wallet_id INT AUTO_INCREMENT PRIMARY KEY,
    player_id INT NOT NULL,
    currency_id INT NOT NULL,
    balance DECIMAL(15, 2) NOT NULL DEFAULT 0.00,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT uq_player_currency UNIQUE (player_id, currency_id),
    CONSTRAINT fk wallets player FOREIGN KEY (player id) REFERENCES players(player id) ON DELETE
    CASCADE,
    CONSTRAINT fk wallets currency FOREIGN KEY (currency id) REFERENCES currencies(currency id) ON
    DELETE RESTRICT
) ENGINE=InnoDB;

-- 4. ITEM CATALOG TABLE
CREATE TABLE item catalog (
    item_type_id INT AUTO_INCREMENT PRIMARY KEY,
    item_name VARCHAR(100) NOT NULL UNIQUE,
    rarity ENUM('Common', 'Uncommon', 'Rare', 'Epic', 'Legendary', 'Mythic') NOT NULL,
    is_tradable BOOLEAN NOT NULL DEFAULT TRUE,
    category ENUM('Assault Rifle', 'SMG', 'Sniper Rifle', 'Combat Helmet', 'Tactical Vest', 'Weapon
    Skin', 'Operator Skin', 'Supply Crate', 'Tactical Gear') NOT NULL,
    base_durability INT NOT NULL DEFAULT 100,
    description TEXT,
    asset_key VARCHAR(50) NOT NULL DEFAULT 'standard_gear'
) ENGINE=InnoDB;

-- 5. INVENTORIES TABLE (Physical Item Instances)
CREATE TABLE inventories (
    item_instance_id INT AUTO_INCREMENT PRIMARY KEY,
    player_id INT NOT NULL,
    item type id INT NOT NULL,
    durability INT NOT NULL DEFAULT 100,
    is equipped BOOLEAN NOT NULL DEFAULT FALSE,
    acquired_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_inv_player FOREIGN KEY (player_id) REFERENCES players(player_id) ON DELETE CASCADE,
```

```
    CONSTRAINT fk_inv_catalog FOREIGN KEY (item_type_id) REFERENCES item_catalog(item_type_id) ON
    DELETE RESTRICT
) ENGINE=InnoDB;

-- 6. STORE OFFERS TABLE
CREATE TABLE store_offers (
    offer_id INT AUTO_INCREMENT PRIMARY KEY,
    item_type_id INT NOT NULL,
    currency_id INT NOT NULL,
    price DECIMAL(15, 2) NOT NULL,
    stock limit INT NOT NULL,
    stock_sold INT NOT NULL DEFAULT 0,
    status ENUM('available', 'sold_out', 'expired', 'disabled') NOT NULL DEFAULT 'available',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_offers_catalog FOREIGN KEY (item_type_id) REFERENCES item_catalog(item_type_id) ON
    DELETE RESTRICT,
    CONSTRAINT fk_offers_currency FOREIGN KEY (currency_id) REFERENCES currencies(currency_id) ON
    DELETE RESTRICT
) ENGINE=InnoDB;

-- 7. MARKET LISTINGS TABLE (P2P Exchange)
CREATE TABLE market listings (
    listing id INT AUTO INCREMENT PRIMARY KEY,
    seller_id INT NOT NULL,
    item_instance_id INT NOT NULL,
    currency_id INT NOT NULL,
    price DECIMAL(15, 2) NOT NULL,
    status ENUM('active', 'sold', 'cancelled', 'expired') NOT NULL DEFAULT 'active',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_market_seller FOREIGN KEY (seller_id) REFERENCES players(player_id) ON DELETE
    CASCADE,
    CONSTRAINT fk_market_item FOREIGN KEY (item_instance_id) REFERENCES inventories(item_instance_id)
    ON DELETE RESTRICT,
    CONSTRAINT fk_market_currency FOREIGN KEY (currency_id) REFERENCES currencies(currency_id) ON
    DELETE RESTRICT
) ENGINE=InnoDB;

-- 8. TRADES TABLE (Bilateral Sessions)
CREATE TABLE trades (
    trade_id INT AUTO_INCREMENT PRIMARY KEY,
    initiator_id INT NOT NULL,
    receiver id INT NOT NULL,
    status ENUM('pending', 'accepted', 'declined', 'cancelled') NOT NULL DEFAULT 'pending',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    resolved_at DATETIME NULL,
    CONSTRAINT fk_trade_initiator FOREIGN KEY (initiator_id) REFERENCES players(player_id) ON DELETE
    CASCADE,
    CONSTRAINT fk_trade_receiver FOREIGN KEY (receiver_id) REFERENCES players(player_id) ON DELETE
    CASCADE
) ENGINE=InnoDB;

-- 9. TRADE ITEMS TABLE
CREATE TABLE trade_items (
    trade_item_id INT AUTO_INCREMENT PRIMARY KEY,
    trade_id INT NOT NULL,
    item_instance_id INT NOT NULL,
    player_id INT NOT NULL,
    CONSTRAINT uq trade item instance UNIQUE (trade id, item instance id),
    CONSTRAINT fk_ti_trade FOREIGN KEY (trade_id) REFERENCES trades(trade_id) ON DELETE CASCADE,
    CONSTRAINT fk_ti_item FOREIGN KEY (item_instance_id) REFERENCES inventories(item_instance_id) ON
    DELETE CASCADE,
    CONSTRAINT fk_ti_player FOREIGN KEY (player_id) REFERENCES players(player_id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 10. TRADE CURRENCIES TABLE
CREATE TABLE trade_currencies (
    trade_currency_id INT AUTO_INCREMENT PRIMARY KEY,
    trade_id INT NOT NULL,
    currency_id INT NOT NULL,
    player id INT NOT NULL,
    amount DECIMAL(15, 2) NOT NULL,
    CONSTRAINT uq_trade_player_currency UNIQUE (trade_id, player_id, currency_id),
    CONSTRAINT fk_tc_trade FOREIGN KEY (trade_id) REFERENCES trades(trade_id) ON DELETE CASCADE,
```

```
    CONSTRAINT fk_tc_currency FOREIGN KEY (currency_id) REFERENCES currencies(currency_id) ON DELETE
    RESTRICT,
    CONSTRAINT fk_tc_player FOREIGN KEY (player_id) REFERENCES players(player_id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 11. AUDIT LOGS TABLE
CREATE TABLE audit_logs (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    player_id INT NULL,
    action_type VARCHAR(50) NOT NULL,
    details TEXT NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_audit_player FOREIGN KEY (player_id) REFERENCES players(player_id) ON DELETE SET
    NULL
) ENGINE=InnoDB;

-- 12. WHEEL EVENTS TABLE (Luck Royale Promotional Engine)
CREATE TABLE wheel_events (
    event_id INT AUTO_INCREMENT PRIMARY KEY,
    event_name VARCHAR(100) NOT NULL,
    currency_id INT NOT NULL,
    spin cost DECIMAL(15, 2) NOT NULL DEFAULT 500.00,
    is active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_wheel_currency FOREIGN KEY (currency_id) REFERENCES currencies(currency_id) ON
    DELETE RESTRICT
) ENGINE=InnoDB;

-- 13. WHEEL REWARDS TABLE (Atomic Scarcity Cap Engine)
CREATE TABLE wheel_rewards (
    reward_id INT AUTO_INCREMENT PRIMARY KEY,
    event id INT NOT NULL DEFAULT 1,
    item_type_id INT NOT NULL,
    base_probability DECIMAL(6, 4) NOT NULL,
    max_supply INT NULL,
    awarded_count INT NOT NULL DEFAULT 0,
    is active BOOLEAN NOT NULL DEFAULT TRUE,
    display order INT NOT NULL DEFAULT 0,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_wr_catalog FOREIGN KEY (item_type_id) REFERENCES item_catalog(item_type_id) ON
    DELETE RESTRICT,
    CONSTRAINT fk_wr_event FOREIGN KEY (event_id) REFERENCES wheel_events(event_id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 14. WHEEL SPINS TABLE (Non-Repudiation & Idempotency)
CREATE TABLE wheel_spins (
    spin_id INT AUTO_INCREMENT PRIMARY KEY,
    player id INT NOT NULL,
    reward_id INT NOT NULL,
    item_instance_id INT NOT NULL,
    currency_id INT NOT NULL,
    cost_paid DECIMAL(15, 2) NOT NULL,
    idempotency_key VARCHAR(64) NULL UNIQUE,
    spun_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_ws_player FOREIGN KEY (player_id) REFERENCES players(player_id) ON DELETE CASCADE,
    CONSTRAINT fk_ws_reward FOREIGN KEY (reward_id) REFERENCES wheel_rewards(reward_id) ON DELETE
    RESTRICT,
    CONSTRAINT fk_ws_item FOREIGN KEY (item_instance_id) REFERENCES inventories(item_instance_id) ON
    DELETE CASCADE
) ENGINE=InnoDB;

-- 15. RELATIONAL VIEW: INVENTORY WITH MARKET STATUS

CREATE OR REPLACE VIEW v player inventory details AS
SELECT
    i.item_instance_id,
    i.player_id,
    p.username AS owner_username,
    c.item_type_id,
    c.item_name,
    c.rarity,
    c.category,
    c.is_tradable,
    c.asset_key,
```

```
    i.durability,
    i.is_equipped,
    i.acquired_at,
    CASE
        WHEN ml.listing_id IS NOT NULL THEN 'LISTED'
        ELSE 'AVAILABLE'
    END AS availability_status,
    ml.listing_id AS active_listing_id,
    ml.price AS listing_price
FROM inventories i
JOIN players p ON i.player id = p.player id
JOIN item_catalog c ON i.item_type_id = c.item_type_id
LEFT JOIN market_listings ml ON i.item_instance_id = ml.item_instance_id AND ml.status = 'active';
```

6. Complete DML & Seed Dataset

To provide a fully populated, production-grade testbed, the database is seeded with 128+ interconnected records including 8 active operators, 4 currencies, 16 wallets, 17 catalog blueprints, 24 inventory items, 10 store offers, 10 market listings, 6 bilateral trades, and 9 Luck Royale reward probabilities.

```
-- ============================================================================
-- BLACKSITE // OPERATOR ARMORY - SEED DATASET
-- ============================================================================
USE blacksite_armory;

-- 1. PLAYERS (8 Operators)
INSERT INTO players (player_id, username, email, status, created_at) VALUES
    (1, 'GHOST ACTUAL', 'ghost@blacksite.mil', 'active', '2026-01-10 08:00:00'),
    (2, 'VIPER SEVEN', 'viper@blacksite.mil', 'active', '2026-01-12 10:15:00'),
    (3, 'SHADOW LEAD', 'shadow@blacksite.mil', 'active', '2026-01-15 14:30:00'),
    (4, 'VALKYRIE_TAC', 'valk@blacksite.mil', 'active', '2026-01-18 09:20:00'),
    (5, 'NOMAD_SNIPER', 'nomad@blacksite.mil', 'active', '2026-01-20 16:45:00'),
    (6, 'TITAN_HEAVY', 'titan@blacksite.mil', 'active', '2026-01-22 11:10:00'),
    (7, 'RECON_SPECTRE', 'recon@blacksite.mil', 'suspended', '2026-01-25 12:00:00'),
    (8, 'CYPHER BREACH', 'cypher@blacksite.mil', 'active', '2026-02-01 17:05:00');

-- 2. CURRENCIES (4 Denominations)
INSERT INTO currencies (currency_id, currency_name, is_premium, symbol, description) VALUES
    (1, 'Combat Credits', FALSE, 'CR', 'Standard tactical military operational currency awarded for
    combat missions.'),
    (2, 'Blacksite Tokens', TRUE, 'BST', 'Encrypted elite tier tokens used for black market
    prototypes and armory drops.'),
    (3, 'War Bonds', FALSE, 'WB', 'Standard veteran service scrip redeemable at tactical supply
    depots.'),
    (4, 'Crypto-Scrip', TRUE, 'CSP', 'High-security anonymous digital currency for high-stakes
    marketplace trade.');

-- 3. WALLETS (16 Accounts)
INSERT INTO wallets (wallet id, player id, currency id, balance, updated at) VALUES
    (1, 1, 1, 45200.00, NOW()),
    (2, 1, 2, 1850.00, NOW()),
    (3, 2, 1, 31400.00, NOW()),
    (4, 2, 2, 2400.00, NOW()),
    (5, 3, 1, 12800.00, NOW()),
    (6, 3, 2, 650.00, NOW()),
    (7, 4, 1, 89500.00, NOW()),
    (8, 4, 2, 5200.00, NOW()),
    (9, 5, 1, 7600.00, NOW()),
    (10, 5, 2, 300.00, NOW()),
    (11, 6, 1, 54000.00, NOW()),
    (12, 6, 2, 1100.00, NOW()),
    (13, 7, 1, 1200.00, NOW()),
    (14, 7, 2, 50.00, NOW()),
    (15, 8, 1, 26700.00, NOW()),
    (16, 8, 2, 3350.00, NOW());

-- 4. ITEM CATALOG (17 Blueprints)
INSERT INTO item_catalog (item_type_id, item_name, rarity, is_tradable, category, base_durability,
    description, asset_key) VALUES
    (1, 'Phantom Circuit', 'Legendary', TRUE, 'Assault Rifle', 100, 'M4A1 composite frame coated with
    electromagnetic circuit patterns.', 'rifle phantom'),
    (2, 'Nightwatch', 'Epic', TRUE, 'Operator Skin', 100, 'Night-recon balaclava and stealth ghillie
    rig with silenced webbing.', 'skin_nightwatch'),
    (3, 'Obsidian Vector', 'Mythic', TRUE, 'SMG', 100, 'Compact high-fire rate SMG milled from
    volcanic obsidian polymer.', 'smg_vector'),
    (4, 'Ghost Protocol', 'Rare', TRUE, 'Combat Helmet', 95, 'Ballistic carbon weave helmet fitted
    with quad-nod NVG brackets.', 'helmet ghost'),
    (5, 'Ironclad', 'Epic', TRUE, 'Tactical Vest', 90, 'Ceramic level-IV multi-strike plate carrier
    with quick-release buckles.', 'vest_ironclad'),
    (6, 'Redline Spectre', 'Legendary', TRUE, 'Weapon Skin', 100, 'Matte gunmetal finish accented
    with aggressive crimson telemetry stripes.', 'skin_redline'),
    (7, 'Recon Pulse', 'Uncommon', TRUE, 'Tactical Gear', 85, 'Handheld RF wide band scanner for
    identifying hostile comm relays.', 'gear_pulse'),
```

```
    (8, 'Blackout Cache', 'Rare', TRUE, 'Supply Crate', 100, 'Mil-spec sealed munitions crate
    containing classified prototype blueprints.', 'crate_blackout'),
    (9, 'Vanguard M24', 'Rare', TRUE, 'Sniper Rifle', 90, 'Precision bolt action 7.62mm chassis
    optimized for extreme range.', 'sniper_vanguard'),
    (10, 'Apex Viper', 'Epic', TRUE, 'Assault Rifle', 95, 'Gas-piston bullpup carbine tuned for rapid
    target transition.', 'rifle_apex'),
    (11, 'Shadow Walker', 'Common', TRUE, 'Tactical Gear', 80, 'Low-signature acoustic dampening
    boots with reinforced steel shanks.', 'gear_boots'),
    (12, 'Aegis Sentinel', 'Rare', TRUE, 'Tactical Vest', 85, 'Lightweight patrol vest with
    integrated hydration pack and chest rig.', 'vest_aegis'),
    (13, 'Cryo Shock', 'Mythic', TRUE, 'Weapon Skin', 100, 'Superconducting liquid-nitrogen pulse
    chamber visual effect skin.', 'skin_cryo'),
    (14, 'SpecOps Breacher', 'Common', TRUE, 'Combat Helmet', 75, 'Standard issue fast-cut ballistic
    helmet with ear protection rails.', 'helmet_standard'),
    (15, 'Havoc Carbine', 'Uncommon', TRUE, 'SMG', 85, 'Close-quarters compact SMG with side-folding
    skeleton stock.', 'smg havoc'),
    (16, 'Founder Token', 'Legendary', FALSE, 'Tactical Gear', 100, 'Honorary non tradable
    commendation awarded to elite alpha testers.', 'gear_medal'),
    (17, 'Phantom Prototype','Mythic', TRUE, 'Assault Rifle', 100, 'Experimental classified assault
    rifle prototype with superconducting electromagnetic coils. Ultra limited Black Market drop.',
    'rifle_phantom');

-- 5. WHEEL EVENT & REWARDS (Luck Royale Engine)
INSERT INTO wheel_events (event_id, event_name, currency_id, spin_cost, is_active) VALUES
    (1, 'BLACK MARKET // LUCK ROYALE', 1, 500.00, TRUE);
INSERT INTO wheel_rewards (reward_id, event_id, item_type_id, base_probability, max_supply,
    awarded_count, is_active, display_order) VALUES
    (1, 1, 17, 0.0050, 2, 0, TRUE, 1),  -- Phantom Prototype (0.5%, Cap: 2)
    (2, 1, 3, 0.0150, NULL, 0, TRUE, 2),  -- Obsidian Vector (1.5%)
    (3, 1, 1, 0.0400, NULL, 0, TRUE, 3),  -- Phantom Circuit (4.0%)
    (4, 1, 6, 0.0600, NULL, 0, TRUE, 4),  -- Redline Spectre (6.0%)
    (5, 1, 2, 0.1200, NULL, 0, TRUE, 5),  -- Nightwatch (12.0%)
    (6, 1, 10, 0.1600, NULL, 0, TRUE, 6),  -- Apex Viper (16.0%)
    (7, 1, 9, 0.2000, NULL, 0, TRUE, 7),  -- Vanguard M24 (20.0%)
    (8, 1, 15, 0.2000, NULL, 0, TRUE, 8),  -- Havoc Carbine (20.0%)
    (9, 1, 11, 0.2000, NULL, 0, TRUE, 9);  -- Shadow Walker (20.0%)
```

7. Some Related Queries

1. Retrieve all registered tactical operators who have an "active" account status, displaying their player ID, username, email address, and registration timestamp, ordered alphabetically by username.

```
SELECT player_id, username, email, status, created_at
FROM players
WHERE status = 'active'
ORDER BY username ASC;
```

2. Find all tactical item blueprints from the catalog that are either of "Mythic" or "Legendary" rarity AND have a base durability of at least 95, excluding items that are designated as non-tradable.

```
SELECT item_type_id, item_name, rarity, category, base_durability, is_tradable
FROM item_catalog
WHERE (rarity = 'Mythic' OR rarity = 'Legendary')
  AND base durability >= 95
  AND is tradable = 1
ORDER BY base_durability DESC, rarity ASC;
```

3. Retrieve all weapons and tactical equipment whose name starts with "Phantom" or contains the word "Viper" anywhere in the item name.

```
SELECT item_type_id, item_name, category, rarity, asset_key
FROM item_catalog
WHERE item name LIKE 'Phantom%' OR item name LIKE '%Viper%'
ORDER BY item_name ASC;
```

4. Retrieve all physical inventory item instances whose current durability falls between 80% and 95% inclusive, displaying the instance ID, owner player ID, item type ID, and current durability, sorted from highest to lowest durability.

```
SELECT item_instance_id, player_id, item_type_id, durability, is_equipped, acquired_at
FROM inventories
WHERE durability BETWEEN 80 AND 95
ORDER BY durability DESC;
```

5. Find all Armory Store offers whose featured item belongs to defensive equipment categories ("Combat Helmet", "Tactical Vest", "Tactical Gear") and whose price is NOT denominated in currency #4 (Crypto-Scrip).

```
SELECT so.offer_id, c.item_name, c.category, c.rarity, so.price, so.currency_id, so.status
FROM store_offers so
JOIN item_catalog c ON so.item_type_id = c.item_type_id
WHERE c.category IN ('Combat Helmet', 'Tactical Vest', 'Tactical Gear')
  AND so.currency_id NOT IN (4)
ORDER BY so.price ASC;
```

6. List all bilateral trades in the system, showing trade ID, initiator ID, receiver ID, status, and the resolution timestamp, displaying "STILL_PENDING" if the trade has not yet been resolved.

```
SELECT trade_id, initiator_id, receiver_id, status, created_at,
       COALESCE(CAST(resolved at AS CHAR), 'STILL PENDING') AS resolution status
FROM trades
ORDER BY trade_id ASC;
```

7. Calculate system-wide summary metrics for all physical inventory items: total count of instantiated items, average durability, lowest durability, and peak durability across all player inventories.

```
SELECT
    COUNT(*) AS total_inventory_items,
    ROUND(AVG(durability), 2) AS avg_fleet_durability,
    MIN(durability) AS lowest durability,
    MAX(durability) AS peak_durability,
    SUM(CASE WHEN is_equipped = 1 THEN 1 ELSE 0 END) AS total_equipped_items
FROM inventories;
```

8. Group all catalog items by their equipment category and determine the number of blueprints in each category, along with the average base durability and the highest durability in that category.

```
SELECT
    category,
    COUNT(*) AS total_blueprints,
    ROUND(AVG(base durability), 1) AS avg base durability,
    MAX(base durability) AS max base durability
FROM item_catalog
GROUP BY category
ORDER BY total_blueprints DESC, category ASC;
```

9. Find all item rarity tiers that have at least 2 distinct physical item instances instantiated in player inventories AND have an average durability strictly greater than 85%.

```
SELECT
    c.rarity,
    COUNT(i.item instance id) AS total instances,
    ROUND(AVG(i.durability), 2) AS avg_durability,
    MIN(i.durability) AS min_durability
FROM inventories i
JOIN item_catalog c ON i.item_type_id = c.item_type_id
GROUP BY c.rarity
HAVING COUNT(i.item instance id) >= 2 AND AVG(i.durability) > 85.00
ORDER BY total_instances DESC;
```

10. Display a complete inspection of all currently equipped loadout items, joining players, physical inventories, and item blueprints to show the operator username, blueprint name, rarity, category, and instance durability.

```
SELECT
    p.player id,
    p.username AS operator_handle,
    c.item_name,
    c.rarity,
    c.category,
    i.durability AS current durability,
    i.acquired at
FROM inventories i
INNER JOIN players p ON i.player_id = p.player_id
INNER JOIN item_catalog c ON i.item_type_id = c.item_type_id
WHERE i.is_equipped = 1
ORDER BY p.player_id ASC, c.rarity DESC;
```

11. Retrieve all registered operators and their active market listings, including operators who currently have zero active listings on the Black Market (showing NULL for listing columns).

```
SELECT
    p.player_id,
    p.username,
    p.status AS account_status,
    ml.listing_id,
    ml.price AS asking price,
    ml.status AS listing_status
FROM players p
LEFT JOIN market_listings ml ON p.player_id = ml.seller_id AND ml.status = 'active'
ORDER BY p.player_id ASC;
```

12. Show all item blueprint types defined in the catalog and any store offers currently offering that item type, ensuring blueprint archetypes that are not featured in the store are still preserved.

```
SELECT
    so.offer_id,
    c.item_type_id,
    c.item name,
    c.category,
    c.rarity,
    so.price,
    so.status AS offer status
FROM store_offers so
RIGHT JOIN item catalog c ON so.item type id = c.item type id
ORDER BY c.item_type_id ASC
LIMIT 12;
```

13. Compare active market listings selling the same currency denomination to identify price spreads where Listing A is priced lower than Listing B.

```
SELECT
    m1.listing id AS listing cheaper id,
    m1.seller id AS seller a,
    m1.price AS cheaper_price,
    m2.listing_id AS listing_expensive_id,
    m2.seller_id AS seller_b,
    m2.price AS expensive_price,
    (m2.price - m1.price) AS price difference,
    m1.currency_id
FROM market_listings m1
JOIN market_listings m2 ON m1.currency_id = m2.currency_id
  AND m1.price < m2.price
  AND m1.status = 'active'
  AND m2.status = 'active'
ORDER BY price_difference DESC
LIMIT 5;
```

14. Retrieve all physical inventory item instances whose current durability is less than their catalog base durability by joining inventories with the item catalog.

```
SELECT
    i.item_instance_id,
    i.player id,
    c.item name,
    c.rarity,
    i.durability,
    c.base_durability
FROM inventories i
JOIN item catalog c ON i.item type id = c.item type id
WHERE i.durability < c.base_durability
ORDER BY i.durability DESC;
```

15. Retrieve all operators who possess a Combat Credits (currency #1) wallet balance that is greater than the average Combat Credits balance across all operators in the database.

```
SELECT
    p.player_id,
    p.username,
    w.balance AS combat_credits_balance,
    (SELECT ROUND(AVG(w_sub.balance), 2)
     FROM wallets w_sub WHERE w_sub.currency_id = 1) AS avg_credits_all_players
FROM players p
JOIN wallets w ON p.player_id = w.player_id
WHERE w.currency id = 1
  AND w.balance > (
      SELECT AVG(w2.balance)
      FROM wallets w2
      WHERE w2.currency_id = 1
  )
ORDER BY w.balance DESC;
```

16. Identify all operators who have NEVER created a market listing on the Black Market, using the NOT EXISTS clause.

```
SELECT p.player_id, p.username, p.email, p.status
FROM players p
WHERE NOT EXISTS (
    SELECT 1
    FROM market_listings ml
    WHERE ml.seller_id = p.player_id
)
ORDER BY p.player_id ASC;
```

17. Find all store offers priced in Combat Credits (currency #1) whose price is cheaper than ANY active marketplace listing denominated in Combat Credits.

```
SELECT
    so.offer_id,
    c.item_name,
    c.rarity,
    so.price AS store price,
    so.stock_limit - so.stock_sold AS stock_left
FROM store_offers so
JOIN item_catalog c ON so.item_type_id = c.item_type_id
WHERE so.currency_id = 1
  AND so.price < ANY (
      SELECT ml.price
      FROM market listings ml
      WHERE ml.currency_id = 1 AND ml.status = 'active'
  )
ORDER BY so.price ASC;
```

18. Construct a consolidated operational log of all economic touchpoints in the system by taking the UNION of all sellers on the Black Market and all initiators of bilateral trades.

```
SELECT seller_id AS operator_id, 'MARKET_SELLER' AS activity_role, created_at AS event_timestamp
FROM market_listings
UNION
SELECT initiator_id AS operator_id, 'TRADE_INITIATOR' AS activity_role, created_at AS event_timestamp
FROM trades
ORDER BY event_timestamp DESC
LIMIT 8;
```

19. Retrieve all active rewards on the Luck Royale wheel along with their event name, spin cost, item name, rarity, base probability, and awarded count by joining wheel rewards, wheel events, and the item catalog.

```
SELECT
    wr.reward id,
    we.event_name,
    we.spin_cost,
    c.item_name,
    c.rarity,
    wr.base_probability,
    wr.max supply,
    wr.awarded count
FROM wheel_rewards wr
JOIN wheel events we ON wr.event id = we.event id
JOIN item_catalog c ON wr.item_type_id = c.item_type_id
WHERE wr.is_active = 1
ORDER BY wr.display_order ASC;
```

20. Restore the durability of all equipped items to 100 for operators who have an "active" account status by joining inventories and players, followed by a verification query.

```
UPDATE inventories i
JOIN players p ON i.player id = p.player id
SET i.durability = 100
WHERE i.is_equipped = 1
  AND p.status = 'active';

SELECT
    i.item instance id,
    p.username,
    c.item_name,
    c.category,
    i.durability AS updated_durability,
    i.is equipped
FROM inventories i
JOIN players p ON i.player_id = p.player_id
JOIN item_catalog c ON i.item_type_id = c.item_type_id
WHERE i.is_equipped = 1
ORDER BY i.item instance id ASC
LIMIT 6;
```

8. Core Subsystems & Business Logic

The BLACKSITE system is divided into five main subsystems. Each subsystem has a specific purpose and helps the game run smoothly, maintain fairness, and protect the database from errors and cheating.

8.1 Player Equipment and Loadouts

The Armory stores all the weapons and armor owned by players. To prevent cheating and item duplication, each item can only be equipped by one character at a time. If a player wants to use the same item on another character, they must first unequip it from the current character. This keeps the equipment system fair and prevents the same item from being used in multiple places at once.

8.2 Supply Depot & Vendor Purchasing

The Supply Depot handles the purchase of limited-edition items, such as rare weapons with only 500 copies available. When many players try to buy an item at the same time, the database processes their requests in a controlled order. It checks the player's balance, deducts the required amount, adds the item to their inventory, and reduces the available stock by one. Once all 500 copies are sold, the item is marked as sold out. This prevents overselling and ensures players are only charged when their purchase is successful.

8.3 Scarcity-Driven P2P Black Market

The Black Market allows players to buy and sell items directly through a public marketplace. When a player lists an item for sale, it is removed from their active inventory and locked until the sale is completed or cancelled. This prevents the seller from using or trading the item while it is listed. When another player buys it, the database transfers the item to the buyer and deducts the money from their wallet in the same transaction. This helps prevent scams and ensures that both the payment and item transfer happen safely.

8.4 Direct Player Trading

The Escrow System allows two players to trade items and currency securely. Both players place their items or money into a temporary trading window, where they can inspect the offers before accepting. The trade is completed only when both players agree to the final deal. If either player changes their offer before the trade is completed, the system cancels the transaction. This prevents last-second changes and helps make player-to-player trading safe and fair.

8.5 Scarcity-Driven Luck Royale & Redistribution Engine

The Luck Royale is a spin-to-win feature that uses a probability system to keep rewards fair. If a limited reward, such as a rare weapon with only two copies available, runs out, it is removed from the reward wheel. The system then recalculates the probabilities and redistributes the unavailable reward's chances among the remaining rewards. This means the chances of winning other available items increase as rare rewards are claimed.

Before awarding a reward, the database checks whether the item has reached its maximum supply. It uses a secure transaction to prevent the reward from being given out more times than allowed. The

probability redistribution formula also ensures that the chances of winning all available rewards add up to 100%.

Overall, these five subsystems help BLACKSITE manage player equipment, purchases, marketplace sales, private trades, and reward distribution. By using secure database transactions and proper stock management, the system reduces cheating, prevents duplicate items, and provides a fair experience for all players.

9. Concurrency Control & Transaction Management

A core requirement of enterprise DBMS design is the preservation of ACID properties (Atomicity, Consistency, Isolation, Durability) in the presence of concurrent requests.

9.1 Deadlock Prevention Strategy

BLACKSITE prevents circular deadlocks using a Global Resource Ordering protocol. All player wallets are locked in ascending order of player_id, ensuring that every transaction follows the same locking sequence. This prevents transactions from waiting for each other in a circular manner. Since a transaction cannot request a lower-ID wallet after locking a higher-ID wallet, circular deadlocks are mathematically eliminated.

```
-- Deadlock-Free Locking Protocol
const firstId = Math.min(sellerId, buyerId);
const secondId = Math.max(sellerId, buyerId);
await client.query('SELECT * FROM wallets WHERE player_id = $1 FOR UPDATE', [firstId]);
await client.query('SELECT * FROM wallets WHERE player_id = $2 FOR UPDATE', [secondId]);
```

9.2 Network Idempotency Tokens

In online games, a laggy internet connection might make you panic and click the "Buy" or "Spin" button multiple times. In a poorly designed system, mashing that button could accidentally charge you twice for the exact same item.

To completely stop this from happening, our system attaches a hidden, one-time "ticket number" to every single click you make. When the database receives your purchase request, it processes the sale and crosses that ticket number off its list.

If your game lags and accidentally sends that exact same ticket number a second or third time, the system immediately recognizes it as a duplicate. Instead of charging your wallet again, it safely ignores the extra clicks and just shows you the item you already bought. This guarantees that you will never be double-charged, no matter how aggressively you press the button.

10. Security, Dual-Gateway Access & Audit Trails

1. The Player View (Tactical Terminal)

This is what regular players see when they log into the game. You can manage your personal gear, visit the store, trade with friends, and spin the prize wheel. However, you are strictly locked to your own

account. You cannot snoop around in other players' private information, and it is physically impossible for you to hack the system to give yourself free money or change the rules of the game.

2. The Developer View (Command Center)

This is the master control room for the game's creators and administrators. They get a live, behind-the-scenes dashboard where they can see exactly what is happening across the entire game world. From this control panel, the developers can design brand-new weapons, add new currency into the economy, or reset the limited items on the prize wheel.

3. The Permanent Security Log

To stop hackers, catch cheaters, and track down bugs, the game keeps an indestructible digital receipt book. Every single time anyone buys an item, finishes a trade, or spins the prize wheel, the system writes down exactly who did it, what they got, and the exact second it happened. These records are permanent and can never be erased, meaning every single item and coin in the game can be tracked perfectly from the moment it was created.

11. Overall Takeaway / Conclusion

The **BLACKSITE // ARMORY** project successfully implements a relational database management system tailored for virtual item economy. By translating conceptual domain requirements into an 11-table relational architecture governed by strict Third Normal Form (3NF) compliance and formal functional dependencies, the system eliminates data redundancy and prevents update, insertion, and deletion anomalies.

At the implementation level, the integration of modular stored procedures, dynamic search and filter routines, and transaction-safe concurrency controls demonstrates effective database-layer enforcement of complex business logic. These mechanisms ensure absolute atomicity and data integrity across critical system workflows, including multi-currency wallet management, developer store transactions, open marketplace listings, and multi-party trade exchanges.